

速记 50 个高频知识点

1、逆向工程

(1) 逆向工程的四个层级：

实现级：包括程序的抽象语法树、符号表、过程的设计表示。

结构级：包括反映程序分量之间相互依赖关系的信息，例如调用图、结构图、程序和数据结构。

功能级：包括反映程序段功能及程序段之间关系的信息，例如数据和控制流模型。

领域级：包括反映程序分量或程序诸实体与应用领域概念之间对应关系的信息，例如实体关系模型。

(2) 逆向工程实现方法

分析信息交换过程；反汇编；反编译。

(3) 与逆向工程相关的概念

重构 (restructuring)：重构是指在同一抽象级别上转换系统描述形式。

设计恢复 (design recovery)：设计恢复是指借助工具从已有程序中抽象出有关数据设计、总体结构设计和过程设计等方面的信息。

逆向工程 (reverse engineering)：逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是设计的恢复过程。

正向工程 (forward engineering)：正向工程是指不仅从现有系统中恢复设计信息，而且使用该信息去改变或重构现有系统，以改善其整体质量。

再工程 (re-engineering)：再工程是对现有系统的重新开发过程，包括逆向工程、新需求的考虑过程和正向工程三个步骤。

2、面向对象设计原则

单一职责原则：设计目的单一的类。

开放-封闭原则：对扩展开放，对修改封闭。

李氏 (Liskov) 替换原则：子类可以替换父类。

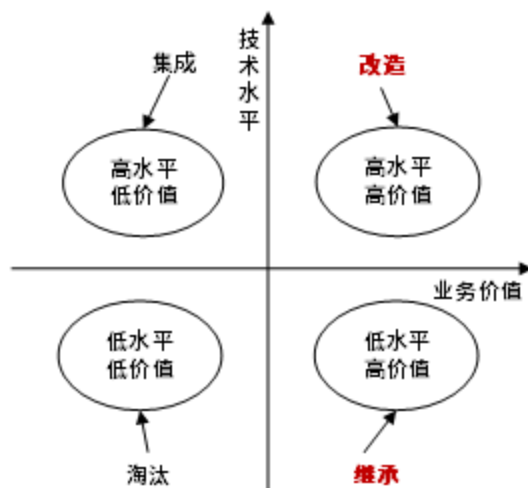
依赖倒置原则：要依赖于抽象，而不是具体实现；针对接口编程，不要针对实现编程。

接口隔离原则：使用多个专门的接口比使用单一的总接口要好。

组合重用原则：要尽量使用组合，而不是继承关系达到重用目的。

迪米特 (Demeter) 原则 (最少知识法则)：一个对象应当对其他对象有尽可能少的了解。

3、遗留系统演化策略



淘汰策略：遗留系统的技术含量较低，且具有较低的业务价值。对遗留系统的完全淘汰是企业资源的根本浪费，系统分析师应该善于“变废为宝”，通过对遗留系统功能的理解和借鉴，可以帮助新系统的设计，降低新系统开发的风险。

继承策略：遗留系统的技术含量较低，已经满足企业运作的功能或性能要求，但具有较高的商业价值，目前企业的业务尚紧密依赖该系统。对这种遗留系统的演化策略为继承。在开发新系统时，需要完全兼容遗留系统的功能模型和数据模型。为了保证业务的连续性，新老系统必须并行运行一段时间，再逐渐切换到新系统上运行。

改造策略：遗留系统具有较高的业务价值，基本上能够满足企业业务运作和决策支持的需要。这种系统可能建成的时间还很短，对这种遗留系统的演化策略为改造。改造包括系统功能的增强和数据模型的改造两个方面。系统功能的增强是指在原有系统的基础上增加新的应用要求，对遗留系统本身不做改变；数据模型的改造是指将遗留系统的旧的数据模型向新的数据模型的转化。

集成策略：遗留系统的技术含量较高，但其业务价值较低，可能只完成某个部门（或子公司）的业务管理。这种系统在各自的局部领域里工作良好，但对于整个企业来说，存在多个这样的系统，不同的系统基于不同的平台、不同的数据模型，形成了一个信息孤岛，对这种遗留系统的演化策略为集成。

4、创建型设计模式

创建型模式与对象的创建有关，抽象了实例化过程，它们帮助一个系统独立于如何创建、组合和表示它的那些对象。一个类创建型模式使用继承改变被实例化的类，而一个对象创建型模式将实例化委托给另一个对象。

设计模式名称	简要说明	速记关键字
Factory Method 工厂方法模式	定义了创建对象的接口，它允许子类决定实例化哪个类	动态生产对象
Abstract Factory 抽象工厂模式	提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定它们具体的类	生产成系列对象

Builder 构建器模式	将一个复杂对象的表示与其构造相分离，使得相同的构建过程能够得出不同的表示	复杂对象构造
Prototype 原型模式	允许对象在不了解要创建对象的确切类以及如何创建等细节的情况下创建自定义对象。通过拷贝原型对象来创建新的对象	克隆对象
Singleton 单例模式	确保一个类只有一个实例，并且提供了对该类的全局访问入口	单实例

5、行为型设计模式

行为模式涉及算法和对象间职责的分配。行为模式不仅描述对象或类的模式，还描述它们之间的通信模式。行为型类模式使用继承机制在类间分配行为，这里包括模板类模式和解释器类模式。行为对象模式使用对象复合而不是继承。一些行为对象模式描述了一组对等的对象怎样相互协作以完成其中任一对象都无法单独完成的任务。

设计模式名称	简要说明	速记关键字
Chain of Responsibility 职责链模式	可以在系统中建立一个链，这样消息可以在首先接收到它的级别处被处理，或者可以定位到可以处理它的对象	传递职责
Command 命令模式	在对象中封装了请求，这样就可以保存命令，将该命令传递给方法以及像任何其他对象一样返回该命令	日志记录，可撤销
Interpreter 解释器模式	可以解释定义其语法表示的语言，还提供了用表示来解释语言中的语句的解释器	虚拟机的机制
Iterator 迭代器模式	为集合中的有序访问提供了一致的方法，而该集合是独立于基础集合，并与之相分离的	数据集
Mediator 中介者模式	通过引入一个能够管理对象间消息分布的对象，简化了系统中对象间的通信	不直接引用
Memento 备忘录模式	保持对象状态的“快照”（snapshot），这样对象可以在不向外界公开其内容的情况下返回到它的最初状态	游戏存档
Observer 观察者模式	为组件向相关接收方广播消息提供了灵活的方法 定义对象间的一种一对多的依赖关系	订阅、广播、联动
State 状态模式	允许一个对象在其内部状态改变时改变它的行为	状态变成类
Strategy 策略模式	定义一系列算法，把它们一个个封装起来，并且使它们之间可互相替换，从而让算法可以独立于使用它的用户而变化	多方案切换
Template Method 模板方法模式	提供了在不重写方法的前提下允许子类重载部分方法的方法	框架
Visitor	提供了一种方便的、可维护的方法来表示在对象结构元素上	数据与操作分离

访问者模式	要进行的操作	
-------	--------	--

6、结构型设计模式

结构型模式处理类或对象的组合，结构型设计模式涉及如何组合类和对象以获得更大的结构。结构型类模式采用继承机制来组合接口或实现。结构型对象模式不是对接口和实现进行组合，而是描述了如何对一些对象进行组合，从而实现新功能的一些方法。

设计模式名称	简要说明	速记关键字
Adapter 适配器模式	将一个类的接口转换成用户希望得到的另一种接口。它使原本不相容的接口得以协同工作	转换接口
Bridge 桥接模式	将一个复杂的组件分成两个独立的但又相关的继承层次结构 将类的抽象部分和它的实现部分分离开来，使它们可以独立地变化	继承树拆分
Composite 组合模式	创建树型层次结构来改变复杂性，同时允许结构中的每一个元素操作同一个接口。用于表示“整体-部分”的层次结构	树形目录结构
Decorator 装饰模式	在不修改对象外观和功能的情况下添加或者删除对象功能 即动态地给一个对象添加一些额外的职责	动态附加职责
Facade 外观模式	子系统中的一组接口提供了一个统一的接口	对外统一接口
Flyweight 享元模式	可以通过共享对象减少系统中低等级的、详细的对象数目 提供支持大量细粒度对象共享的有效方法	汉字编码
Proxy 代理模式	为控制对初始对象的访问提供了一个代理或者占位符对象	快捷方式

7、信息系统开发方法

(1) 结构化开发方法

- 自顶向下，逐步分解求精；
- 严格区分阶段，阶段产出标准化；
- 应变能力差。

(2) 原型法【需求阶段，作用是展示及获取需求】

- 针对需求不明确；
- 按功能分：水平原型（界面）、垂直原型（复杂算法）；
- 按最终结果分：抛弃式原型、演化式原型。

(3) 面向对象方法

- 自底向上；
- 阶段界限不明；
- 更好应变、更好复用；

- 符合人们的思维习惯。

(4) 面向服务的方法

- a) 粗粒度、松耦合；
- b) 标准化和构件化；
- c) 面向服务方法的三个抽象级别：
 - 操作：位于最底层，对标函数方法这个层次。
 - 服务：代表操作的逻辑分组。
 - 业务流程：为实现特定业务目标而执行的一组长期运行的动作或活动。业务流程的例子有：接纳新员工、出售产品或服务并完成订单。

8、I/O 控制方式

(1) 程序控制（查询）方式：分为无条件传送和程序查询方式两种。方法简单，硬件开销小，但 I/O 能力不高，严重影响 CPU 的利用率。

(2) 程序中断方式：与程序控制方式相比，中断方式因为 CPU 无需等待而提高了传输请求的响应速度。

【CPU 与 I/O 传输可并行】

(3) DMA 方式：DMA 方式是为了在主存与外设之间实现高速、批量数据交换而设置的。DMA 方式比程序控制方式与中断方式都高效。【CPU 与 I/O 传输可并行】

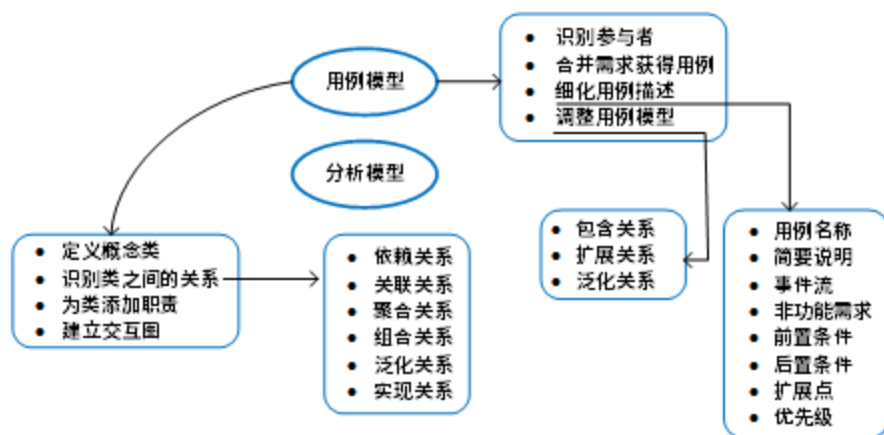
(4) 通道方式

(5) I/O 处理机

9、需求获取方法

方法	特点
用户访谈	1 对 1-3，有代表性的用户，了解主观想法，交互好。成本高，要有领域知识支撑
问卷调查	用户多，无法一一访谈，成本低
采样 (抽样调查)	基于数理统计，降低成本，快速获取 采样技术包括：随机采样、分层采样、聚类采样等，具体采用哪种方法根据文档数量、样本大小和实际情况决定
情节串联板 【原型前身】	一系列图片，通过这些图片来讲故事。类型包括：被动式、主动式和交互式
联合需求计划	高度组织的群体会议，各方参与，了解想法，消除分歧，交互好，成本高
需求记录技术	需求记录工具有：任务卡片、场景说明、用户故事和 Volere 白卡等

10、面向对象分析需求建模



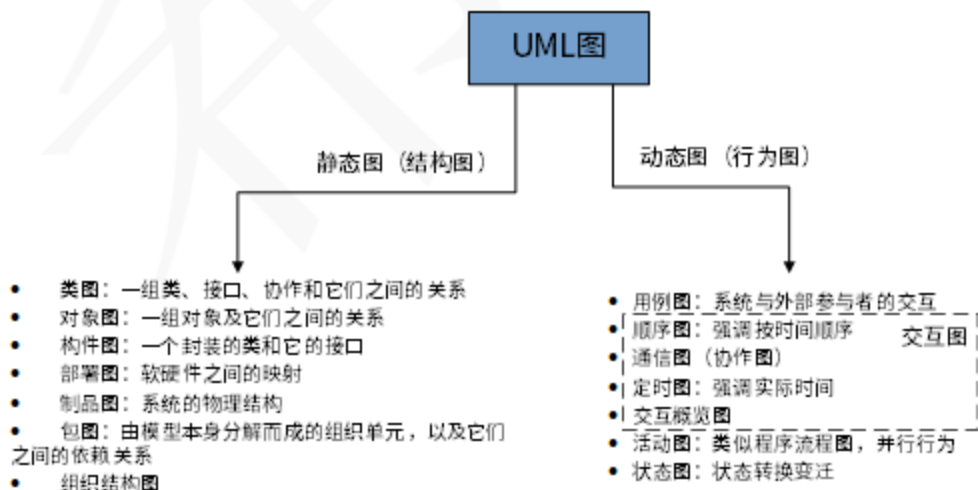
用例模型建立流程：

- 第一步：识别参与者【参与者：用户、组织、外部系统、时间】
- 第二步：合并需求获得用例
- 第三步：细化用例描述
- 第四步：调整用例模型（可选步骤）【关系包括：包含关系、扩展关系、泛化关系】

分析模型建立流程：

- 第一步：定义概念类
- 第二步：确定类之间的关系
- 第三步：为类添加职责
- 第四步：建立交互图
- 第五步：分析模型的详细程度问题

11、UML 的 14 种图



UML 2.0 包括 14 种图，分别如下：

- (1) 类图（class diagram）。类图描述一组类、接口、协作和它们之间的关系。类图给出了系统的静态设计视图，活动类的类图给出了系统的静态进程视图。
- (2) 对象图（object diagram）。对象图描述一组对象及它们之间的关系。对象图描述了在类图所建立的事物实例的静态快照。对象图是从真实案例或原型案例的角度建立的。

(3) 构件图 (component diagram)。构件图描述一个封装的类和它的接口、端口，以及由内嵌的构件和连接件构成的内部结构。构件图用于表示系统的静态设计实现视图。

(4) 组合结构图 (composite structure diagram)。组合结构图描述结构化类（例如，构件或类）的内部结构，包括结构化类与系统其余部分的交互点。组合结构图用于画出结构化类的内部内容。

(5) 用例图 (use case diagram)。用例图描述一组用例、参与者及它们之间的关系。用例图给出系统的静态用例视图。

(6) 顺序图 (sequence diagram, 序列图)。顺序图是一种交互图 (interaction diagram)，交互图展现了一种交互，它由一组对象或参与者以及它们之间可能发送的消息构成。交互图专注于系统的动态视图。顺序图是强调消息的时间次序的交互图。

(7) 通信图 (communication diagram)。通信图也是一种交互图，它强调收发消息的对象或参与者的结构组织。

(8) 定时图 (timing diagram, 计时图)。定时图也是一种交互图，它强调消息跨越不同对象或参与者的实际时间，而不仅仅只是关心消息的相对顺序。

(9) 状态图 (state diagram)。状态图描述一个状态机，它由状态、转移、事件和活动组成。状态图给出了对象的动态视图。它对于接口、类或协作的行为建模尤为重要，而且它强调事件导致的对象行为，这非常有助于对反应式系统建模。

(10) 活动图 (activity diagram)。活动图将进程或其它计算结构展示为计算内部一步步的控制流和数据流。活动图专注于系统的动态视图。它对系统的功能建模和业务流程建模特别重要，并强调对象间的控制流程。

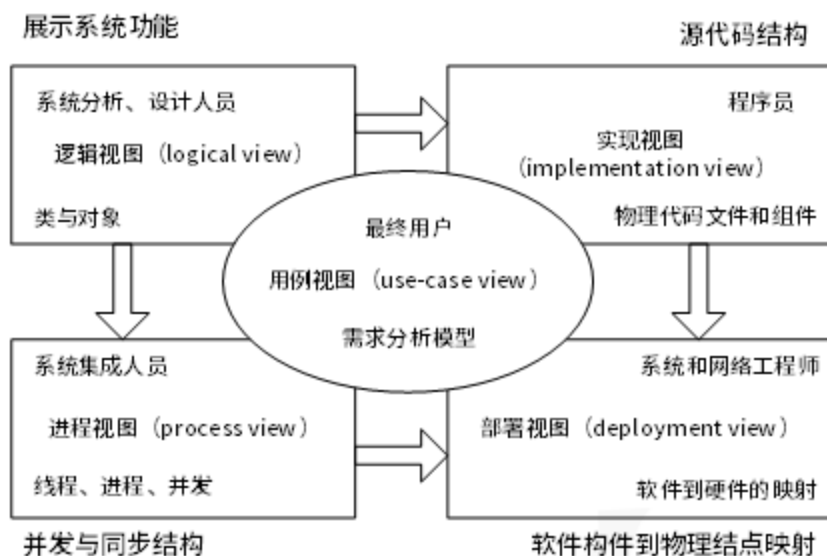
(11) 部署图 (deployment diagram)。部署图描述对运行时的处理节点及在其中生存的构件的配置。部署图给出了架构的静态部署视图，通常一个节点包含一个或多个部署图。

(12) 制品图 (artifact diagram)。制品图描述计算机中一个系统的物理结构。制品包括文件、数据库和类似的物理比特集合。制品图通常与部署图一起使用。制品也给出了它们实现的类和构件。

(13) 包图 (package diagram)。包图描述由模型本身分解而成的组织单元，以及它们之间的依赖关系。

(14) 交互概览图 (interaction overview diagram)。交互概览图是活动图和顺序图的混合物。

12、UML 4+1 视图



(1) 逻辑视图：以问题域的语汇组成的类和对象集合。

(2) 进程视图：可执行线程和进程作为活动类的建模，它是逻辑视图的一次执行实例，描绘了所设计的并发与同步结构。

(3) 实现视图：对组成基于系统的物理代码的文件和组件进行建模。

(4) 部署视图：把构件部署到一组物理的、可计算的节点上，表示软件到硬件的映射及分布结构。

(5) 用例视图：最基本的需求分析模型。

13、NoSQL 分类

类型	典型产品	典型应用场景	数据模型	优点	缺点
Key-Value	Memcached、Redis	缓存、日志等	key-value/全键值对	扩展性好，灵活性强，大量操作时性能高	数据无结构化
列式存储	BigTable、HBase、Cassandra	分布式数据存储和管理	列簇式存储	可扩展性强，查找速度快，复杂度低	功能相对局限，不支持事务强一致性
文档型	MongoDB、CouchDB	Web 应用	key-value, value 为结构化或半结构化数据	结构灵活，可以根据 value 构建索引	查询性能不高、缺乏统一语言
图数据库	Neo4j、OrientDB	社交网络、推荐系统等	图存储三个字段：节点，关系和属性	支持复杂的图形算法	复杂度高，只能支持一定数据规模，实现困难

14、可行性研究分类

(1) 经济可行性：成本收益分析，包括建设成本、运行成本和项目建设后可能的经济收益。

(2) 技术可行性：技术风险分析，现有的技术能否支持系统目标的实现，现有资源（员工、技术积累、构件库、软硬件条件）是否足以支持项目的实施。

(3) 法律可行性【社会可行性】：不能与国家法律或政策相抵触。

(4) 用户使用可行性：从信息系统用户的角度评估系统的可行性。

管理可行性：系统与现有管理机制的一致性，改革的可能性。

运行可行性：用户方便使用的程度，及用户的素质和培训要求。

15、五大架构风格

五大架构风格	子风格	核心特点及注意事项
数据流风格 Data Flow	批处理 Batch Sequential 管道-过滤器 Pipes and Filters	数据驱动，适合用于前一步处理结果作为后一步数据处理输入的场景。
调用/返回风格 Call/Return	主程序/子程序 Main Program and Subroutine 面向对象 Object-oriented 分层架构 Layered System	层次型就属于这种，通过分层做内容的解耦。如：网络七层模型，B/S。
独立构件风格 Independent Components	进程通信 Communicating Processes 事件驱动系统（隐式调用） Event system	强调不直接调用，事件触发是典型。如：订阅-发布模式。
虚拟机风格 Virtual Machine	解释器 interpreter 规则系统 Rule-based System	用于自定义场景。能不写代码自定义业务流程并执行。如：解释器。
以数据为中心 Data-centered	数据库系统 Database System 黑板系统 Blackboard System 超文本系统 Hypertext System	各种构件围绕【数据中心】工作。如：数据库系统、黑板风格。

16、微服务

(1) 概念

微服务顾名思义，就是很小的服务，所以它属于面向服务架构的一种。

微服务在更小的基础上，其实进一步在突显其独立性。

(2) 微服务的优势

- 小服务【且专注于做一件事情】
- 轻量级的通信机制
- 松耦合
- 独立测试
- 独立部署【简单部署】
- 独立运行【每个服务独立在其独立进程中】

- 支持异构【如每个服务使用不同数据库】
- 更好的可用性与弹性
- 化整为零，易于小团队开发

(3) 微服务面临的挑战

分布式环境下的数据一致性【更复杂】

测试的复杂性【服务间依赖测试】

运维的复杂性

(4) 微服务架构原则

单一职责原则：每个微服务只关注一个业务领域。

隔离性原则：每个微服务的故障不影响其它服务运行。

自治性原则：每个微服务是自治的，具有自己的生命周期、状态和行为。

弹性原则：可快速适应不断变化的负载和需求。

可观察性原则：微服务是可观察的，可以收集和分析其状态和行为数据。

可测试性原则：微服务可测试，用自动化测试和集成测试实现。

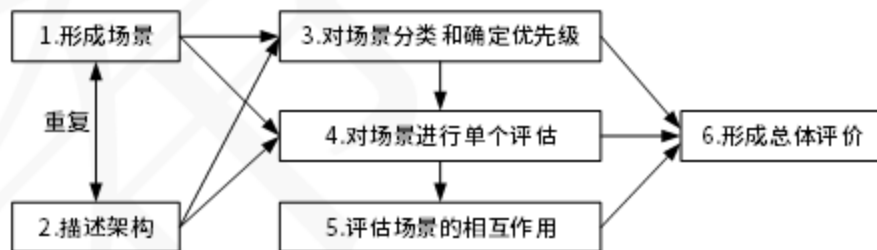
可维护性原则：微服务可以进行快速维护和修改。

安全性原则：通过加密、认证和授权技术来实现。

智能端点与简单消息传递原则：服务端点尽可能智能和独立，服务之间通信用简单消息传递。

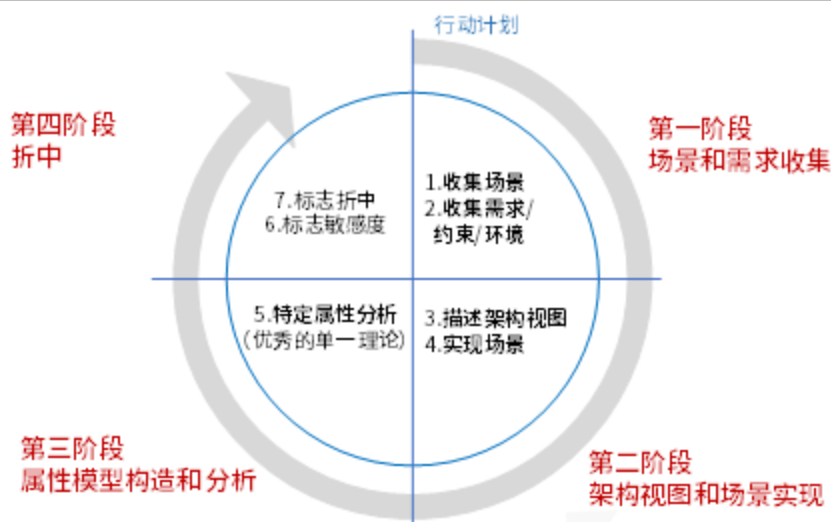
17、架构评估方法---SAAM 和 ATTM

(1) SAAM：最初用于分析架构可修改性，后扩展到其他质量属性。



(2) ATAM：在 SAAM 的基础上发展起来的。ATAM 希望揭示出架构满足特定质量目标的情况，使架构设计师更清楚地认识到质量目标之间的联系，即如何权衡多个质量目标。

主要针对性能、可用性、安全性和可修改性。



18、主从数据库

(1) 主从数据库结构特点：

一般：一主多从，也可以多主多从。

主库做写操作，从库做读操作。

(2) 主从复制步骤：

主库（Master）更新数据完成前，将操作写入 binlog 日志文件。

从库（Slave）打开 I/O 线程与主库连接，做 binlog dump process，并将事件写入中继日志。

从库执行中继日志事件，保持与主库一致。

19、电子政务

电子政务主要有 3 类角色：政府（Government）、企（事）业单位（Business）及公民（Citizen）。

如果有第 4 类就是公务员（Employee）。

类型	应用
G2G	基础信息的采集、处理和利用，如：人口信息、地理信息 各级政府决策支持
G2E	政府内部管理系统
G2B	政府给企业单位颁发【各种营业执照、许可证、合格证、质量认证】
B2G	企业向政府缴税 企业向政府供应各种商品和服务【含竞/投标】 企业向政府提建议，申诉
G2C	社区公安和水、火、天灾等与公共安全有关的信息 户口、各种证件和牌照的管理
C2G	个人应向政府缴纳的各种税款和费用 个人向政府反馈民意【征求群众意见】

20、嵌入式实时操作系统调度算法分类

(1) 基于优先级的抢占调度：基于优先级的抢占调度在任何时候运行的任务都是所有就绪任务中具有最高优先级的任务。优先级可以分为静态优先级和动态优先级。

(2) 时间轮转调度算法：为每个任务提供确定份额的 CPU 执行时间。

(3) 单调速率调度 (Rate Monotonic Scheduling, RMS) 算法：静态固定优先级算法，任务周期越短优先级越高，是静态调度中最优的。

(4) 截止时间优先 (Earliest Deadline First, EDF) 调度算法 (最早截止期调度算法)：根据任务的截止时间来确定其优先级，对于时间期限最近的任务，分配最高的优先级。在动态调度算法中 CPU 的利用率达到了最高。

(5) 最低松弛度优先 (Least Laxity First, LLF) 算法：根据任务紧急 (或松弛) 的程度，来确定任务的优先级。任务的紧急程度愈高，为该任务所赋予的优先级就愈高，使之优先执行。动态调度算法。

(6) 基于分区 (partition) 化的分区 APPS 调度算法：将处理器时间资源划分成多个独立时间片资源，以供多个分区实体独立享用。通过遵循特定的分区调度原则和时间优先调度 (TPS) 策略，该算法可以确保系统的稳定性和性能。静态调度算法。

21、工作流参考模型



(1) 工作流执行服务：核心模块，功能包括创建和管理流程定义，创建、管理和执行流程实例。

(2) 工作流引擎：是为流程实例提供运行环境，并解释执行流程实例的软件模块。

(3) 流程定义工具：是管理流程定义的工具，它可以通过图形方式把复杂的流程定义显示出来并加以操作，流程定义工具与工作流执行服务交互，一般该模块为设计人员提供图形化的用户界面。

(4) 客户端应用：是通过请求的方式与工作流执行服务交互的应用。

(5) 调用应用：是被工作流执行服务调用的应用，调用应用与工作流执行服务交互。

(6) 管理监控工具：主要指组织机构和参与者等数据的维护管理和流程执行情况的监控，管理监控工具与工作流执行服务交互。

22、企业资源计划（ERP）

（1）发展过程



企业资源：支持企业业务活动和战略运营的事物

- 人、财、物和信息资源
- 三流：物流、资金流和信息流

（2）ERP 结构

管理思想：它是管理思想的变革。

软件产品：不是直接买来就用，需要个性化的开发与部署。

管理系统：存在众多的子系统，这些子系统有统一的规划，是互联互通的，便于事前事中监控。

◆ 财会管理

- 会计核算
- 财务管理

◆ 物流管理

- 分销管理
- 库存控制
- 采购管理

◆ 生产控制管理

- 主生产计划
- 物料需求计划
- 能力需求计划
- 车间控制
- 制造标准

◆ 人力资源管理

- 人力资源规划
- 招聘管理
- 工资核算
- 工时管理
- 差旅费核算

23、内聚

内聚：衡量模块内部各元素结合的紧密程度。

功能内聚：完成一个单一功能，各个部分协同工作，缺一不可。

顺序内聚：处理元素相关，而且必须顺序执行。

通信内聚：所有处理元素集中在一个数据结构的区域上。

过程内聚：处理元素相关，而且必须按特定的次序执行。

瞬时内聚（时间内聚）：所包含的任务必须在同一时间间隔内执行。

逻辑内聚：完成逻辑上相关的一组任务。

偶然内聚（巧合内聚）：完成一组没有关系或松散关系的任务。

24、耦合

耦合：度量不同模块间互相依赖的程度。

非直接耦合：两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的。

数据耦合：一组模块借助参数表传递简单数据。

标记耦合：一组模块通过参数表传递记录信息（数据结构）。

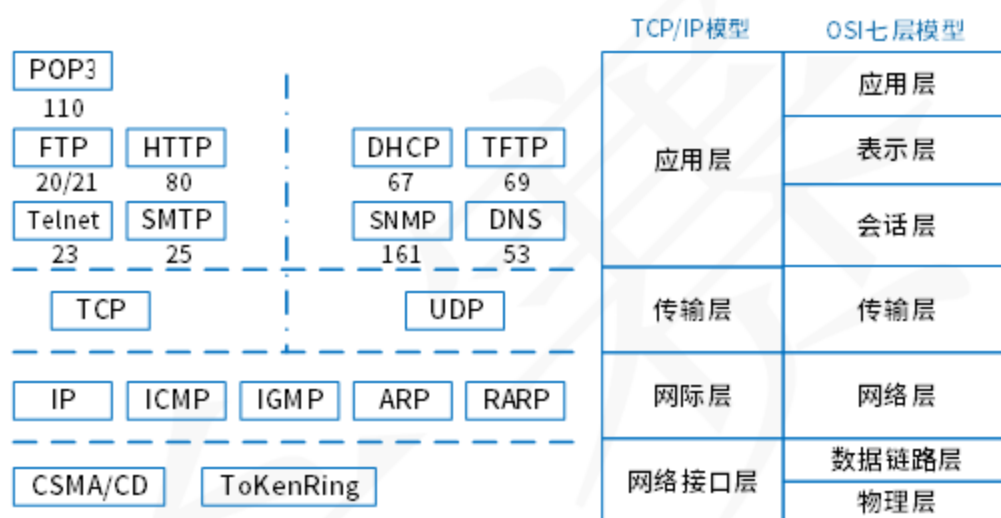
控制耦合：模块之间传递的信息中包含用于控制模块内部逻辑的信息。

外部耦合：一组模块都访问同一全局简单变量，而且不是通过参数表传递该全局变量的信息。

公共耦合：多个模块都访问同一个公共数据环境。

内容耦合：一个模块直接访问另一个模块的内部数据；一个模块不通过正常入口转到另一个模块的内部；两个模块有一部分程序代码重叠；一个模块有多个入口。

25、TCP/IP 协议簇



常见协议介绍：

TCP 与 UDP 均支持对具体指定端口号进行通信。但连接管理、差错校验、重传等能力只有 TCP 具备。

TCP：可靠的传输层协议

UDP：不可靠的传输层协议

DHCP：67 端口，IP 地址自动分配

SNMP：161 端口，简单网络管理协议

ICMP：因特网控制协议，PING 命令来自该协议

IGMP：组播协议

Telnet：23 端口，远程协议。（不安全，SSH 是安全的远程协议）

HTTP：80 端口，超文本传输协议，网页传输

DNS：53 端口，域名解析协议，记录域名与 IP 的映射关系

ARP：地址解析协议，IP 地址转换为 MAC 地址

RARP：反向地址解析协议，MAC 地址转 IP 地址

FTP：20 数据端口/21 控制端口，文件传输协议

POP3：110 端口，邮件收取

SMTP: 25 端口，邮件发送

IMAP 和 POP3 的区别是：POP3 协议允许电子邮件客户端下载服务器上的邮件，但是在客户端的操作（如移动邮件、标记已读等），不会反馈到服务器上，比如通过客户端收取了邮箱中的 3 封邮件并移动到其它文件夹，邮箱服务器上的这些邮件是没有同时被移动的。而 IMAP 客户端的操作都会反馈到服务器上，对邮件进行的操作，服务器上的邮件也会做相应的动作。

MIME 是多用途互联网邮件扩展标准，与邮件安全无关。MIME/S 与邮件安全相关。

26、系统维护分类

正确性维护【修 BUG】：识别和纠正软件错误/缺陷，测试不可能发现所有错误。

适应性维护【应变】：指使应用软件适应环境变化【外部环境、数据环境】而进行的修改。

完善性维护【新需求】：扩充功能和改善性能而进行的修改。

预防性维护【针对未来】：为了适应未来的软硬件环境的变化，应主动增加预防性的新的功能，以使应用系统适应各类变化而不被淘汰。经典实例：【专用】改【通用】。

27、负载均衡算法

（1）静态算法（不考虑动态负载）

轮转算法：轮流将服务请求（任务）调度给不同的节点（即：服务器）。

加权轮转算法：考虑不同节点处理能力的差异。

源地址哈希散列算法：根据请求的源 IP 地址，作为散列键从静态分配的散列表找出对应的节点。

目标地址哈希散列算法：根据请求目标 IP 做散列找出对应节点。

随机算法：随机分配，简单，但不可控。

（2）动态算法（考虑动态负载）

最小连接数算法：每个节点处理能力相同的情况下，新请求分配给当前活动请求数量最少的节点。

加权最小连接数算法：考虑节点处理能力不同，按最小连接数分配。

加权百分比算法：考虑了节点的利用率、硬盘速率、进程个数等，使用利用率来表现剩余处理能力。

28、企业集成

（1）按集成点分：

	集成点	效果	解题关键点
界面集成	界面	统一入口，产生“整体”感觉	“整体”感觉 最小代价实现一体化操作
数据集成	数据	不同来源的数据逻辑或物理上“集中”	其他集成方法的基础
控制集成	应用逻辑	调用其他系统已有方法，达到集成效果	/
业务流程集成 (过程集成)	应用逻辑	跨企业，或优化流程而非直接调用	企业之间的信息共享能力

门户集成	/	将内部系统对接到互联网上	发布到互联网上
------	---	--------------	---------

(2) 按传输方式划分：

	特点
消息集成	数据量小，交互频繁，立即地，异步
共享数据库	交互频繁，立即地，同步
文件传输	数据量大，交互频度小，即时性要求低（月末，年末）

29、同步调用与异步调用

(1) 同步调用 (Synchronous)

调用方发出请求后，必须原地等待服务方处理完毕并返回结果，才能继续执行后续的代码。

(2) 异步调用 (Asynchronous)

调用方发出请求后，不需要等待结果，直接去执行后续的任务。结果通常通过回调 (Callback)、消息队列 (MQ) 或通知来异步处理。

(3) 同步调用 VS 异步调用

特性	同步调用	异步调用
等待结果	是 (阻塞)	否 (非阻塞)
响应速度	取决于最慢的那个服务	极快 (通常直接返回接收成功)
系统复杂性	低	高
适用场景	强依赖结果 (如：银行转账、用户登录)	耗时且不需结果 (如：发送邮件、日志收集)

30、大数据处理系统

(1) 概念

大数据是具有数量巨大、来源多样、生成极快、多变等特征且难以使用传统数据体系结构有效处理的包含大量数据集的数据

(2) 大数据的 5 个 V：

- 大规模 (Volume)
- 高速度 (Velocity)
- 多样化 (Variety)
- 价值密度低 (Value)
- 真实性 (Veracity)

(3) 通用设计原则

- 可扩展 (分布式、模块化)
- 可管理 (管理策略和方法)
- 数据安全 (数据加密、访问控制、数据备份与恢复、审计日志、持续监控)
- 高性能 (数据分区、分布式计算、并行处理、内存计算)

高可用（数据冗余、自动故障转移、负载均衡）

稳定性（监控和预警、性能调优、故障排除、升级和维护）

31、工作分解结构【WBS】

（1）WBS 的作用

便于估算，明确范围，防止需求蔓延。最底层是工作包。

（2）范围基准是经过批准的范围说明书、WBS 和相应的 WBS 词典。

（3）WBS 工作分解注意事项

- WBS 必须是面向可交付成果的；
- 必须符合项目范围，在各层次上保持项目的完整性，避免遗漏必要的组成部分（100%规则）；
- 底层工作应该支持计划和控制，具有可比性，是可管理的，可定量检查的（8-80 小时规则）；
- 工作单元应能分开不同的责任者和不同工作内容（每个工作单元有且仅有一个人负责）；
- 一个工作单元只能从属于某个上层单元，避免交叉从属，4-6 层；
- 应包括项目管理工作，也包括分包出去的工作；
- WBS 的编制需要所有（主要）项目干系人的参与；
- WBS 并非是一成不变的（滚动分解原则）。

32、Redis 分布式存储方式

分布式存储方案	核心特点
主从（Master/Slave）模式	一主多从，故障时手动切换。
哨兵（Sentinel）模式	有哨兵的一主多从，主节点故障自动选择新的主节点。
集群（Cluster）模式	分节点对等集群，分 slots，不同 slots 的信息存储到不同节点。

33、Redis 持久化

Redis 的持久化主要有两种方式：RDB 和 AOF

RDB：传统数据库中快照的思想。指定时间间隔将数据进行快照存储。

AOF：传统数据库中日志的思想，把每条改变数据集的命令追加到 AOF 文件末尾，这样出问题了，可以重新执行 AOF 文件中的命令来重建数据集。

对比维度	RDB 持久化	AOF 持久化
备份量	重量级的全量备份，保存整个数据库	轻量级增量备份，一次只保存一个修改命令
保存间隔时间	保存间隔时间长	保存间隔时间短，默认 1 秒
还原速度	数据还原速度快	数据还原速度慢
阻塞情况	save 会阻塞，但 bgsave 或者自动不会阻塞	无论是平时还是 AOF 重写，都不会阻塞
数据体积	同等数据体积：小	同等数据体积：大

安全性	数据安全性：低，容易丢数据	数据安全性：高，根据策略决定
-----	---------------	----------------

34、REST

(1) 概念：REST (Representational State Transfer, 表述性状态转移) 是一种通常使用 HTTP 和 XML 进行基于 Web 通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

(2) REST 的 5 个原则

网络上的所有事物都被抽象为资源。

每个资源对应一个唯一的资源标识（反过来不成立）。

通过通用的连接件接口对资源进行操作。

对资源的各种操作不会改变资源标识。

所有的操作都是无状态的。

(3) REST API

命令	用途
GET	从指定的资源请求数据，只进行数据检索，不进行其他操作。
POST	将数据发送到服务器进行创建，通常用于上传文件或提交表单。
PUT	更新目标资源的所有当前表示，使用上传的内容进行替换。
DELETE	删除指定的资源。
HEAD	与 GET 方法类似，但只传输状态行和头部信息。
PATCH	对资源进行部分修改。

35、敏捷方法

(1) 传统软件开发方法和敏捷方法的区别

传统软件开发方法	敏捷方法
预设性的	适应性的
以开发过程为本	以人为本
整体分阶段	增量迭代，小步快跑
/	适合小型项目

(2) 概念

敏捷开发是一种以人为核心、迭代、循序渐进的开发方法，适用于小团队和小项目，具有小步快跑的思想。

(3) 敏捷宣言

个体和交互胜过过程和工具；

可工作的软件胜过大量的文档【实现与测试是核心】；

客户合作胜过合同谈判；

响应变化胜过遵循计划。

(4) 敏捷方法

敏捷方法	特点总结
极限编程 (XP)	在一些对费用控制严格的公司中使用，非常有效
水晶方法	探索了用最少纪律约束而仍能成功的方法，从而在产出效率与易于运作上达到一种平衡
开放式源码	程序开发人员在地域上分布很广【其它方法强调集中办公】
SCRUM	明确定义了可重复的方法过程。适用于任何复杂的或创新性的项目
功用驱动开发方法 (FDD)	编程开发人员分成两类：首席程序员和“类”程序员
ASD 方法	其核心是三个非线性的、重叠的开发阶段：猜测、合作与学习
动态系统开发方法 (DSDM)	倡导以业务为核心

36、信息系统战略规划

信息系统战略规划 (Information System Strategic Planning, ISSP) 是从企业战略出发，构建企业基本的信息架构，对企业内、外信息资源进行统一规划、管理与应用，利用信息控制企业行为，辅助企业进行决策，帮助企业实现战略目标。

ISSP 方法经历了三个主要阶段，各个阶段所使用的方法也不一样。

(1) 第一个阶段主要以数据处理为核心，围绕职能部门需求的信息系统规划，主要的方法包括：

企业系统规划法 (BSP, Business System Planning) --CU 矩阵

关键成功因素法 (CSF, Critical Success Factors)

战略集合转化法 (SST, Strategy Set Transformation)

此外还有：投资回收法、征费法、零线预算法、阶石法

(2) 第二个阶段主要以企业内部管理信息系统为核心，围绕企业整体需求进行的信息系统规划，主要的方法包括：

战略数据规划法 (SDP, Strategic Data Planning)：主题数据库

信息工程法 (IE, Information Engineering)

战略栅格法 (SG, Strategic Grid)

(3) 第三个阶段的方法是在综合考虑企业内外环境的情况下，以集成为核心，围绕企业战略需求进行的信息系统规划，主要的方法包括：

价值链分析法 (VCA, Value Chain Analysis)

战略一致性模型 (SAM, Strategic Alignment Model)

37、云计算

(1) 概念

云计算是集合了大量计算设备和资源，对用户屏蔽底层差异的分布式处理架构，其用户与提供实际服务的计算资源是相分离的。

(2) 优点

超大规模、虚拟化、高可靠性、高可伸缩性、按需服务、成本低（前期投入低、综合使用成本也低）

(3) 分类

按服务类型分类：

- 软件即服务（SaaS）：基于多租户技术实现，直接提供应用程序。
- 平台即服务（PaaS）：虚拟中间件服务器、运行环境和操作系统。
- 基础设施即服务（IaaS）：包括服务器、存储和网络等服务。

按部署方式分类：

- 公有云：面向互联网用户需求，通过开放网络提供云计算服务。
- 社区云：云基础设施分配给一些社区组织所专有。
- 私有云：面向企业内部提供云计算服务。
- 混合云：兼顾以上两种情况的云计算服务。

38、测试阶段划分

测试阶段	内容
单元测试	依据详细设计，模块测试，模块功能、性能、接口等
集成测试	依据概要设计，模块间的接口
系统测试	依据需求文档，在真实环境下，验证完整的软件配置项能否和系统正确连接
确认测试	依据需求文档，验证软件与需求的一致性。内部确认测试、Alpha 测试、Beta 测试、验收测试

系统测试又包括：功能测试、性能测试、健壮性测试、用户界面测试、安全性测试、安装与反安装测试等。其中性能测试又可以进一步分类为：

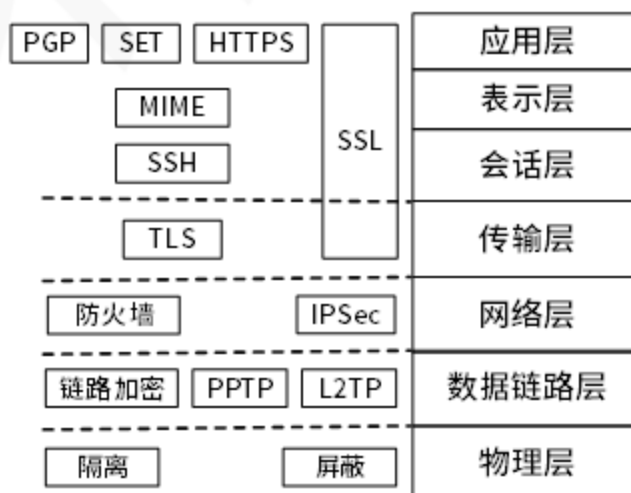
- 负载测试：各种工作负载下系统的性能
- 压力测试【测上限】：系统的瓶颈或不能接受的性能点
- 强度测试【测下限】：系统资源特别低的情况下运行
- 容量测试【并发测试】：同时在线的最大用户数
- 可靠性测试：MTTF 之类的参数

39、知识产权人确定

情况说明	判断说明	归属
作品 职务 作品	利用单位的物质技术条件进行创作，并由单位承担责任的	除署名权外其他著作权归单位
	有合同约定，其著作权属于单位	除署名权外其他著作权归单位

		其他	作者拥有著作权，单位有权在业务范围内优先使用
软件	职务作品	属于本职工作中明确规定的开发目标	单位享有著作权
		属于从事本职工作活动的结果	单位享有著作权
		使用了单位资金、专用设备、未公开的信息等物质、技术条件，并由单位或组织承担责任的软件	单位享有著作权
专利权	职务作品	本职工作中做出的发明创造	单位享有专利权
		履行本单位交付的本职工作之外的任务所做出的发明创造	单位享有专利权
		离职、退休或调动工作后 1 年内，与原单位工作相关	单位享有专利权
作品软件	委托创作	有合同约定，著作权归委托方	委托方
		合同中未约定著作权归属	创作方
	合作开发	只进行组织、提供咨询意见、物质条件或者进行其他辅助工作	不享有著作权
		共同创作的	共同享有，按人头比例。 成果可分割的，可分开申请。
商标		谁先申请谁拥有（除知名商标的非法抢注） 同时申请，则根据谁先使用（需提供证据） 无法提供证据，协商归属，无效时使用抽签（但不可不确定）	
专利		谁先申请谁拥有 同时申请则协商归属，协商不成则同时驳回双方的专利申请	

40、安全协议



PGP (Pretty Good Privacy)：针对邮件和文件的混合加密系统。

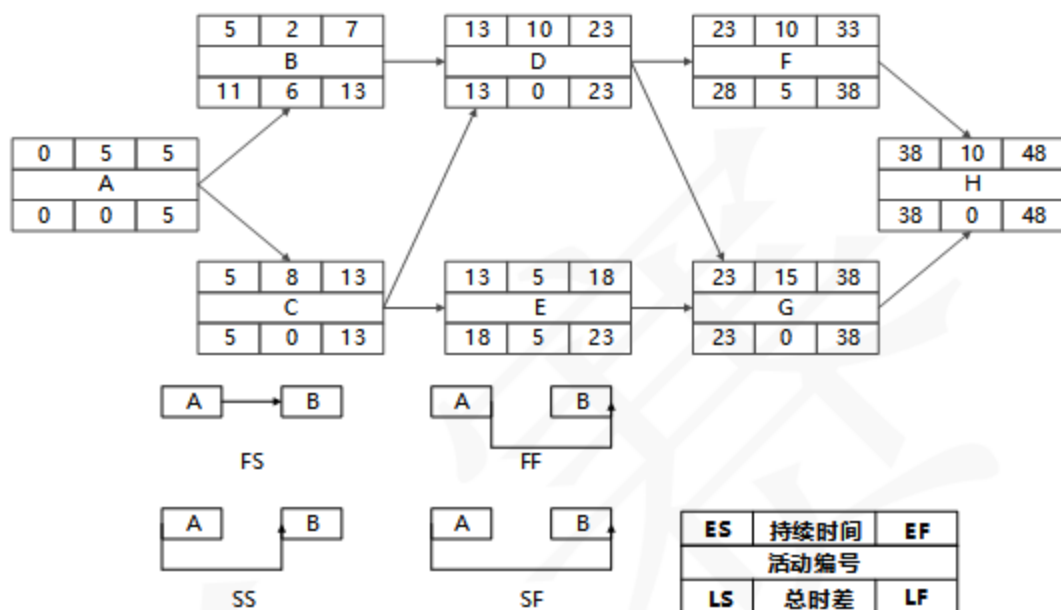
SSL (Secure Sockets Layer) : 工作在传输层至应用层。

TLS (Transport Layer Security) : 传输层安全协议。

SET (Secure Electronic Transaction) : 安全电子交易协议。电子商务，身份认证。普遍的说法是将其归为应用层。

IPsec (Internet Protocol Security) : 对 IP 包加密。

41、PERT 图



ES: 最早开始时间 = 所有紧前活动最早结束时间的最大值。

EF: 最早完成时间 = 最早开始时间 + 持续时间。

LS: 最迟开始时间 = 最迟完成时间 - 持续时间。

LF: 最迟完成时间 = 所有紧后活动最迟开始时间的最小值。

总时差 (松弛时间) : 在不延误总工期的前提下, 该活动的机动时间。活动的总时差等于该活动最迟完成时间与最早完成时间之差, 或该活动最迟开始时间与最早开始时间之差。

42、运维技术指标

(1) 平均故障修复时间 → $MTTR = \frac{\text{【给定时间周期】系统修复总时间}}{\text{维修次数}}$

从出现故障到修复成功中间的这段时间, 衡量系统修复能力

(2) 平均故障间隔时间 → $MTBF = \frac{\text{多次故障之间系统总运行时间}}{\text{故障总数}}$

反映系统运行可靠程度

(3) 平均无故障时间 → $MTTF = \frac{\text{【给定时间周期】正常运行总时间}}{\text{故障总数}}$

系统无故障运行的平均时间, 确定系统或硬件的业务持续性或寿命

(4) 平均应答时间 → $MTTA = \frac{\text{【给定时间周期】系统出现警告到警告确认之间累计总时间}}{\text{事件总数}}$

响应投诉、业务中断、异常事件的平均时间

注: $MTBF = MTTF + MTTR$

43、反规范化

(1) 反规范化的技术手段

增加冗余列是指在多个表中具有相同的列，它常用来在查询时避免连接操作。

增加派生列指增加的列来自其它表中的数据，由它们计算生成。

重新组表指如果许多用户需要查看两个表连接出来的结果数据，则把这两个表重新组成一个表来减少连接而提高性能。

分割表：水平分割与垂直分割。水平分割，有些记录常要查询，有些记录不常用，如历史记录。垂直分割，把主码与某些常用的字段组成一个表，把主码与另一些字段组成另一个表。

(2) 反规范化的优缺点

反规范化的优点：连接操作少，检索快、统计快；需要查的表减少，检索容易。

反规范化的缺点	解决方案
数据冗余，需要更大存储空间	无解
插入、更新、删除操作开销更大	无解
数据不一致 可能产生添加、修改、删除异常	1、触发器数据同步 2、应用程序数据同步 3、批处理
更新和插入代码更难写	无解

44、Lambda 架构和 Kappa 架构

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理，吞吐量大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱
使用场景	直接支持批处理，更适合对历史数据分析查询的场景，期望尽快得到分析结果，批处理可以更直接高效地满足这些需求。	不是 Lambda 的替代架构，而是简化，Kappa 放弃了对批处理的支持，更擅长业务本身为增量数据写入场景的分析需求
选择依据	根据两种架构对比分析，将业务需求、技术要求、系统复杂度、开发维护成本和历史数据处理能力作为选择考虑因素。 计算开销虽然存在一定差别，但是相差不是很大，所以不作为考虑因素。	

45、业务流程分析方法

价值链分析法：找出或设计出那些能够使顾客满意，实现顾客价值最大化的业务流程。

客户关系分析法：把 CRM 用在业务流程的分析上。

供应链分析法：从企业供应链的角度分析企业的业务流程。

基于 ERP 的分析法：将企业的业务流程看作是一个紧密联接的供应链，将供应商和企业内部的采购、生产、销售，以及客户紧密联系起来，对供应链上的所有环节进行有效管理，实现对企业的动态控制和各种资源的集成和优化。

业务流程重组：通过重新审视企业的价值链，从功能成本的比较分析中，确定企业在哪些环节具有比较优势。

46、对称加密

(1) 对称加密（又称为私人密钥加密/共享密钥加密）：加密与解密使用同一密钥。

(2) 特点：加密强度不高，但效率高，易破解；密钥分发困难。

(3) 用途：对消息明文进行加密传送

(4) 常见对称密钥加密算法：

DES：替换+移位、56 位密钥、64 位数据块、速度快、密钥易产生

3DES（三重 DES）：密钥长度 112 位【两个 56 位的密钥 K1、K2】

加密：K1 加密→K2 解密→K1 加密

解密：K1 解密→K2 加密→K1 解密

IDEA：128 位密钥、64 位明文/密文，PGP 采用该算法。

其它：RC-5、AES。

47、非对称加密

(1) 非对称加密（又称为公开密钥加密）：密钥必须成对使用（公钥加密，相应的私钥解密）。

(2) 特点：加密强度高，但效率低，极难破解；密钥分发容易。

(3) 用途：对密钥加密，做数字签名。

(4) 常见非对称密钥加密算法：

RSA：官方教程上是 512 位密钥，不过现在已经支持 2048 位（或 1024 位）密钥。

Elgamal：安全性依赖于计算有限域上离散对数这一难题。

ECC：椭圆曲线算法。

48、区块链

(1) 区块链分类

公有链（Public BlockChains）：无官方组织及管理机构，无中心服务器，参与的节点按照系统规则自由接入网络、不受控制，节点间基于共识机制开展工作。

私有链（Private BlockChains）：建立在某个企业内部，系统的运作规则根据企业要求进行设定，修改甚至是读取权限仅限于少数节点，同时仍保留着区块链的真实性和部分去中心化的特性。

联盟链（Consortium BlockChains）：由若干机构联合发起，介于公有链和私有链之间，兼具部分去中心化的特性。

（2）特点

特点	描述
去中心化	由于使用分布式核算和存储，不存在中心化的硬件或管理机构，任意节点的权利和义务都是均等的，系统中的数据块由整个系统中具有维护功能的节点来共同维护
开放性	系统是开放的，如：区块链上的【交易信息是公开的】，不过【账户身份信息是高度加密的】
自治性	区块链采用基于协商一致的规范和协议（比如一套公开透明的算法），使得整个系统中的所有节点能够在信任的环境自由安全的交换数据，使得对“人”的信任改成了对机器的信任，任何人为的干预不起作用
安全性 (防篡改)	数据在多个节点存储了多份，篡改数据得改掉 51%节点的数据，这太难。同时，还有其它安全机制，如：比特币的每笔交易，都由付款人用私钥签名，证明确实是他同意向某人付款，其他人无法伪造
匿名性 (去信任)	由于节点之间的交换遵循固定的算法，其数据交互是无需信任的（区块链中的程序规则会自行判断活动是否有效），因此交易对手无须通过公开身份的方式让对方对自己产生信任，对信用的累积非常有帮助

（3）区块链的四大基础技术支柱

①分布式账本（数据不存放在单一服务器，而是分散存储在全球数千个节点上）

核心定义：共享、同步、复制的数据库。

核心特性：分布式存储；去中心化；不可篡改性；透明性与可审计性；一致性（最终一致性）

②密码学

哈希函数：将任意数据转换为固定长度字符串（如 SHA-256 算法）。

非对称加密（公私钥签名）：公钥加密（公开地址），私钥解密（仅所有者能操作资产）。

③共识机制（让分散节点对新区块达成一致）

PoW（工作量证明）：比特币采用，节点通过算力竞赛获得记账权（“挖矿”）。

PoS（权益证明）：以太坊采用，持币越多越有机会记账。

④智能合约（自动执行，代码即法律。去中介化不可逆协议）

49、封锁协议

S 封锁：共享锁（S 锁）又称读锁，若事务 T 对数据对象 A 加上 S 锁，其它事务只能再对 A 加 S 锁，而不能加 X 锁，直到 T 释放 A 上的 S 锁。

X 封锁：排他锁（X 锁）又称写锁。若事务 T 对数据对象 A 加上 X 锁，其它事务不能再对 A 加任何锁，直到 T 释放 A 上的锁。

一级封锁协议：事务 T 在修改数据 R 之前必须先对其加 X 锁，直到事务结束才释放。可防止丢失修改。

二级封锁协议：一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁，读完后即可释放 S 锁。可防止丢失修改，还可防止读“脏”数据。

三级封锁协议：一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁，直到事务结束才释放。可防止丢失修改、防止读“脏”数据与防止数据重复读。

两段锁协议：可串行化的。可能发生死锁。

50、事务的特性

原子性（Atomicity）是指事务包含的所有操作要么全部成功，要么全部失败回滚。这些操作是一个整体，不能部分地完成。

一致性（Consistency）是指事务必须使数据库从一个一致性状态变换到另一个一致性状态，也就是说一个事务执行之前和执行之后都必须处于一致性状态。

隔离性（Isolation）是指一个事务的执行不能被其他事务干扰，即一个事务内部的操作及使用的数据对并发的其他事务是隔离的。

持久性（Durability）是指一个事务一旦被提交了，那么对数据库中的数据的改变就是永久性的，无论发生何种故障，都不应对其有任何影响。

制作于 26 年 7 月 适用于第 2 版教材